

Model Interpretation & Communication

Student Engagement Prediction at Tomorrow University

Clarisse Casi Cariño

1. Business Question

Can we predict which students at Tomorrow University are at risk of low academic engagement, using routine learning activity data?

Tomorrow University wants to identify struggling students early — before they drop out or fail — so that academic support teams can intervene proactively. By analysing how students interact with the platform (logins, lessons, posts, assignments, and mentoring), we can flag students who need support before it is too late.

2. Dataset

We used a SQLite database (student_data.db) containing 500 student records. After cleaning and feature selection, the following variables were used:

Feature	Type	Description
logged_in_capped_iqr	Numeric	Hours logged into the platform (outliers capped)
lessons_capped_iqr	Numeric	Number of lessons completed (outliers capped)
posts_capped_iqr	Numeric	Number of forum posts made (outliers capped)
assignments_capped_iqr	Numeric	Assignments submitted (outliers capped)
mentoring_Yes	Binary (0/1)	Whether the student has a mentor (1 = Yes, 0 = No)
score_capped_z	Target (Reg)	Student performance score, standardised
highly_engaged	Target (Clf)	Binary: 1 = above-median engagement, 0 = below

3. Pipeline Overview

A. Data Pre-Processing

- Corrected inconsistent spellings in country, sex, and mentoring columns.
- Imputed missing values using mode (categorical) and median (numeric).
- Removed the birthdate column (perfectly correlated with age, $r = -1.00$) to eliminate redundancy.

- Capped outliers using IQR for logins, lessons, posts, and assignments; Z-score for score.

B. Feature Selection

- Hypothesis testing (Pearson, Point-Biserial, ANOVA) identified 6 significant features.
- Chi-Square confirmed mentoring as the strongest categorical predictor.
- Random Forest SelectFromModel reduced to 5 final features: logins, lessons, posts, assignments, and mentoring.

C. Encoding & Scaling

- Converted mentoring (Yes/No) to binary integer using get_dummies.
- Applied StandardScaler to all numeric features (fit on train only, applied to both sets).

D. Feature Engineering

- total_activity: Sum of all engagement metrics (logins + lessons + assignments + posts).
- ment_act_interact: Interaction term — mentoring status multiplied by total activity.

E. Train-Test Split

- 80% training (400 students), 20% testing (100 students), random_state=42 for reproducibility.

F. Model Selection

- Tested Linear Regression, Decision Tree, and KNN for regression.
- Tested Decision Tree and KNN for classification.
- Applied RandomForestClassifier and RandomForestRegressor with GridSearchCV hyperparameter tuning.

G. Evaluation Metrics

- Regression: R^2 (variance explained) and RMSE (average prediction error).
- Classification: Balanced Accuracy (accounts for class imbalance; 88.8% of students labelled positive).

4. Code Example (Python / scikit-learn)

The following is a simplified, self-contained reproduction of the core modelling pipeline used in this project:

```
# === IMPORTS ===
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import balanced_accuracy_score
from sklearn.pipeline import Pipeline
from sklearn.feature_selection import SelectFromModel

# === FEATURES & TARGET ===
```

```
features = ['logged_in_capped_iqr', 'lessons_capped_iqr',
            'posts_capped_iqr', 'assignments_capped_iqr', 'mentoring_Yes_True']
X = df[features]
y = (df['score_capped_z'] >= df['score_capped_z'].median()).astype(int)

# === TRAIN-TEST SPLIT (80/20) ===
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# === PIPELINE: FEATURE SELECTION + MODEL ===
selector = SelectFromModel(RandomForestClassifier(max_depth=3, random_state=42))
rf = RandomForestClassifier(class_weight='balanced', random_state=42)
pipe = Pipeline([('selector', selector), ('rf', rf)])

# === HYPERPARAMETER TUNING (GridSearchCV) ===
params = {'rf__max_depth': [3, 4], 'rf__n_estimators': [100, 200],
          'rf__min_samples_leaf': [5, 10]}
grid = GridSearchCV(pipe, params, cv=5, scoring='balanced_accuracy')
grid.fit(X_train, y_train)

# === EVALUATION ===
best = grid.best_estimator_
y_pred = best.predict(X_test)
print('Best Params:', grid.best_params_)
print('CV BalAcc:', round(grid.best_score_, 3))
print('Test BalAcc:', round(balanced_accuracy_score(y_test, y_pred), 3))
```

5. Interpretation of Findings

A. Key Findings

Model Performance

Metric	RF Classifier	RF Regressor	Best Model
Test Score	BalAcc: 0.907	R²: 0.728	Classifier ✓
CV Score	0.792	R²: 0.640	Classifier ✓
Train-Test Gap	-0.057	-0.044	Both ✓
Robust CV (10×5)	0.768 ± 0.191	0.636 ± 0.167	Classifier ✓

Key Predictors

Based on Random Forest feature importances, the top drivers of student engagement are:

Feature	Importance Rank	Interpretation
logged_in_capped_iqr	#1 (Highest)	More time logged in = higher engagement
mentoring_Yes	#2	Mentored students score significantly higher
lessons_capped_iqr	#3	Completing more lessons boosts performance

Feature	Importance Rank	Interpretation
posts_capped_iqr	#4	Forum participation linked to engagement
assignments_capped_iqr	#5	Assignment submission shows commitment

Why Classification Over Regression?

Although regression models achieved a solid R^2 of ~ 0.77 , classification was selected as the final approach for the following reasons:

- The dataset has a severe class imbalance (88.8% of students labelled positive). Raw accuracy of 88% is misleading — a model that always predicts “engaged” would score the same without learning anything.
- Balanced accuracy corrects for this imbalance by equally weighting performance on both classes. The tuned classifier achieved a test balanced accuracy of 0.907, well above what a naive classifier would reach.
- Classification directly answers the actionable question: “Is this student at risk?” — which is more useful for academic support teams than a raw score prediction.

B. Use Case Example

The model can be applied as follows in a real-world deployment:

Student Profile	Model Prediction	Recommended Action
0 logins, 0 lessons, no mentor	LOW engagement (0)	Immediate outreach from support team
15 logins, 5 lessons, mentor	HIGH engagement (1)	Continue monitoring, no action needed
3 logins, 1 lesson, no mentor	LOW engagement (0)	Assign mentor, flag for check-in

C. Limitations

- **Small dataset:** With only 500 students, the $\pm$ variation in cross-validation scores (up to ± 0.19) is notable. Results may shift with more data.
- **Class imbalance:** Despite balanced accuracy, the heavily positive-skewed target (88.8%) means the model has seen very few truly disengaged students during training.
- **No temporal data:** The dataset is a static snapshot. Engagement trends over time (e.g., a student who was active but suddenly stopped logging in) could be far more predictive.
- **Self-reported/proxy features:** Features like posts and logins measure activity, not true understanding. A student could post frequently without meaningful engagement.
- **No causal inference:** The model identifies correlation, not causation. Assigning more assignments will not necessarily improve scores.